



université
PARIS-SACLAY

Université d'Évry – Université Paris-Saclay

Master 2 : Computer and Network Systems (CNS)

Module : Apprentissage profond

AST — Audio Spectrogram Transformer

**MIT Computer Science and Artificial
Intelligence Laboratory, Cambridge, MA, USA**

Préparé par : Lakhmi Hichem Billal & Rabah Ouafi

Encadré par : D. Fourer

Année universitaire : 2025–2026

Table des matières

1 Introduction

2 GTZAN Dataset — Classification des genres musicaux

3 Préparation du dataset

3.1 Division en ensembles d'entraînement, de validation et de test

3.2 Découpage en segments

3.3 Conversion en spectrogramme

4 Audio Spectrogram Transformer (AST)

4.1 Architecture du modèle

4.1.1 Préparation de l'entrée

4.1.2 Encodeur et classification

5 Utilisation du modèle avec le dataset GTZAN

5.1 Téléchargement des poids pré-entraînés de l'AST

5.2 Implémentation du modèle et initialisation des paramètres

6 Implémentation du Fine-Tuning de l'Audio Spectrogram Transformer (AST)

6.1 Configuration des Hyperparamètres et Outils

6.1.1 Stratégie du Taux d'Apprentissage

6.2 Fonction d'Évaluation (Validation)

6.3

7 test Accuracy

8 Comparaison entre les modèles CNN et AST

8.1 Principe de fonctionnement des modèles

8.1.1 CNN (Convolutional Neural Network)

8.1.2 AST (Audio Spectrogram Transformer)

8.2 Résultats expérimentaux

8.3	Analyse comparative	
8.4	Avantages et limitations	
8.4.1	Avantages du CNN	
8.4.2	Limitations du CNN	
8.4.3	Avantages du AST	
8.4.4	Limitations du AST	
8.5	Discussion	

1 Introduction

L'objectif de ce projet est de concevoir un système capable de classier automatiquement des extraits musicaux par genre en utilisant le jeu de données GTZAN, qui contient 1000 extraits audio répartis en dix genres : blues, classical, country, disco, hiphop, jazz, metal, pop, reggae et rock.

Deux approches d'apprentissage profond ont été développées, analysées et comparées :

- **AST (Audio Spectrogram Transformer)** — un modèle moderne inspiré des Vision Transformers, appliqué à la représentation spectrogramme du signal audio.
- **CNN classique** — une architecture convolutionnelle standard permettant d'extraire localement les caractéristiques fréquentielles des spectrogrammes.

L'étude détaille la préparation des données, la méthodologie employée pour les deux architectures, puis présente les résultats expérimentaux. Enfin, une comparaison approfondie permet d'évaluer les performances respectives des modèles et leurs avantages et limites.

2 GTZAN Dataset — Classification des genres musicaux

Le dataset GTZAN, considéré comme le « MNIST de l'audio », est utilisé pour la classification des genres musicaux.

Description du dataset :

- **Genres originaux** : 10 genres musicaux avec 100 fichiers audio chacun, d'une durée de 30 secondes.
- **Images originales** : représentation visuelle des fichiers audio sous forme de Mel Spectrograms.
- **Fichiers CSV** : deux fichiers contenant des caractéristiques extraites des fichiers audio. Le premier calcule, pour chaque morceau, la moyenne et la variance de plusieurs descripteurs audio (MFCC, ZCR, spectral centroid, etc.). Le second découpe chaque morceau en segments de 3 secondes, multipliant par dix le nombre d'échantillons pour l'entraînement des modèles.

3 Préparation du dataset

3.1 Division en ensembles d'entraînement, de validation et de test

Le jeu de données a été divisé en trois sous-ensembles pour assurer un apprentissage efficace et une évaluation fiable :

- **Ensemble d'entraînement (Train)** : 699 extraits audio
- **Ensemble de validation (Validation)** : 151 extraits audio
- **Ensemble de test (Test)** : 150 extraits audio

Cette division permet de former le modèle, d'ajuster les hyperparamètres et d'évaluer les performances finales de manière impartiale.

3.2 Découpage en segments

Avant de convertir un fichier audio en spectrogramme, le signal audio est découpé en segments de durée fixe pour garantir une taille constante.

- Le fichier audio est chargé et converti en mono si nécessaire.
- Si la durée est inférieure à celle souhaitée, un *padding* par des zéros est appliqué.
- Si la durée est supérieure, un segment est sélectionné :
 - Pour l'entraînement avec augmentation : extraction aléatoire (*random crop*)
 - Sinon : sélection du segment central

3.3 Conversion en spectrogramme

Chaque segment audio est transformé en *Mel spectrogram*, représentant l'énergie du signal en fonction du temps et de la fréquence.

- Calcul du Mel spectrogram via la transformée de Fourier.
- Conversion de l'amplitude en décibels (*log scale*).
- Normalisation pour obtenir une moyenne nulle et un écart-type unitaire.
- Ajustement de la dimension temporelle par *padding* ou découpage.
- Application optionnelle de *SpecAugment* pour améliorer la diversité des données.

4 Audio Spectrogram Transformer (AST)

L'AST est un modèle de classification audio entièrement basé sur un mécanisme d'attention, inspiré du Vision Transformer (ViT).

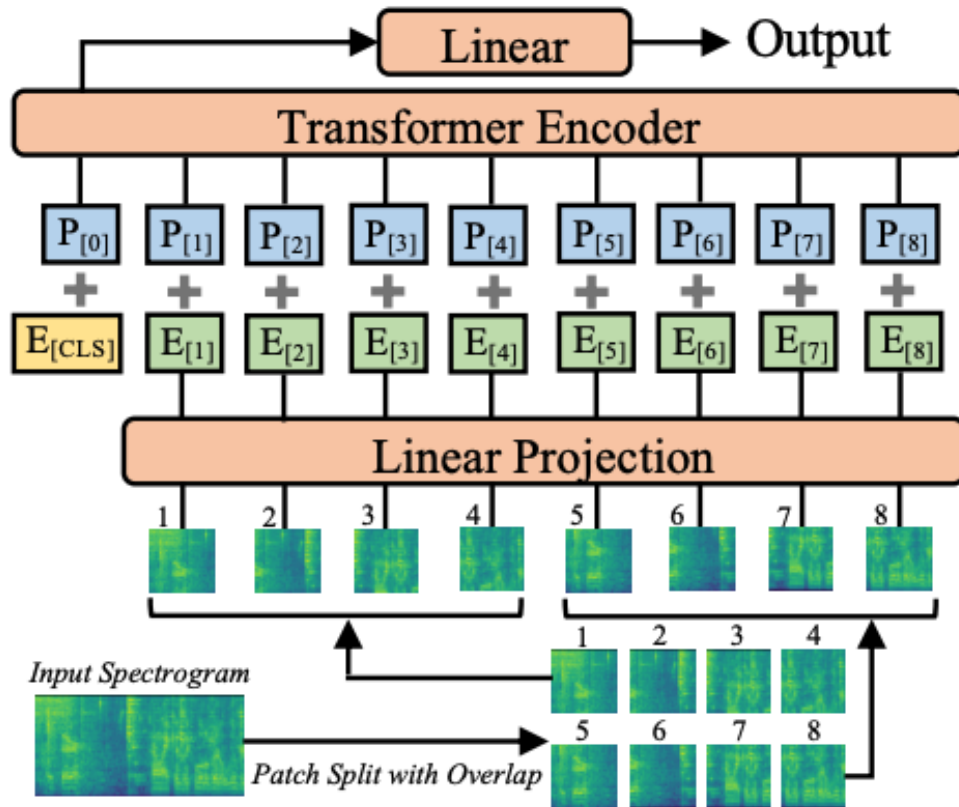


FIGURE 1 – Architecture du modèle AST. Le spectrogramme d’entrée est divisé en patches, projetés, combinés avec des embeddings de position, puis traités par l’Encodeur Transformer.

4.1 Architecture du modèle

Le modèle prend en entrée un spectrogramme audio 2D et applique l’architecture standard de l’Encodeur Transformer.

4.1.1 Préparation de l’entrée

- **Spectrogramme Mel-log** : forme d’onde audio convertie en spectrogramme Mel-log de dimension 128.
- **Découpage en patches** : spectrogramme divisé en patches de 16×16 .
- **Chevauchement** : overlap de 6 sur dimensions temporelle et fréquentielle.
- **Projection linéaire** : chaque patch projeté en vecteur 1D de taille 768.
- **Embedding de position** : embedding trainable ajouté à chaque patch.
- **Token [CLS]** : token ajouté pour la classification.

4.1.2 Encodeur et classification

- **Encodeur Transformer** : 12 couches, 12 têtes d’attention, embedding 768.
- **Représentation** : sortie du token [CLS].

— **Classification** : couche linéaire avec sigmoid.

5 Utilisation du modèle avec le dataset GTZAN

5.1 Téléchargement des poids pré-entraînés de l'AST

```
1 import os
2
3
4 real_pretrained_dir = "/content/drive/MyDrive/ast/pretrained_models"
5
6
7 !mkdir -p /content/../../
8 !ln -sfn "{real_pretrained_dir}" /content/../../pretrained_models
9
```

FIGURE 2 – Téléchargement des poids pré-entraînés du modèle AST.

Pour profiter des performances maximales de l'AST, nous utilisons des poids pré-entraînés sur de grands jeux de données tels qu'ImageNet et AudioSet. Cela permet d'éviter un entraînement complet depuis zéro et d'obtenir des représentations audio robustes dès le départ. Ces poids sont téléchargés automatiquement via la bibliothèque utilisée et servent de point de départ pour l'entraînement sur le dataset GTZAN.

5.2 Implémentation du modèle et initialisation des paramètres

```
1 from ast_models import ASTModel
2
3
4 model = ASTModel(
5     label_dim=NUM_CLASSES,
6     fstride=10, tstride=10,
7     input_fdim=INPUT_FDIM,
8     input_tdim=INPUT_TDIM,
9     imagenet_pretrain=True,
10    audioset_pretrain=True,
11    model_size="base384",
12    verbose=True
13 ).to(DEVICE)
14
15
16 for name, param in model.named_parameters():
17     if ("blocks" not in name) and ("mlp_head" not in name):
18         param.requires_grad = False
19
20 trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
21 total = sum(p.numel() for p in model.parameters())
22 print(f"Trainable: {trainable}/{total}")
23
```

FIGURE 3 – Initialisation du modèle AST avec les paramètres et poids pré-entraînés.

Le modèle AST est instancié avec les dimensions des entrées et le nombre de classes correspondant à notre dataset. Les poids pré-entraînés sont chargés, et seuls certains paramètres sont entraînaables, afin de conserver les connaissances apprises lors du pré-entraînement.

Code Python utilisé :

Explication du code :

- **ASTModel** : Classe du modèle AST, prenant en entrée les dimensions des spectrogrammes et le nombre de classes.
- **fstride et tstride** : Paramètres de stride sur les dimensions fréquentielles et temporelles pour le découpage en patches.
- **input_fdim et input_tdim** : Dimensions des spectrogrammes en fréquence et temps.
- **imagenet_pretrain et audioset_pretrain** : Chargement des poids pré-entraînés pour accélérer l'entraînement et améliorer la performance.
- **model_size** : Taille du modèle (ici "base384" pour la version de base adaptée aux spectrogrammes 384).
- **Verrouillage des paramètres** : Seules les couches "blocks" et "mlp_head" sont entraînaables, les autres gardent les poids pré-entraînés pour préserver les connaissances.
- **Nombre de paramètres** : Calcul du total et du nombre de paramètres entraînaables pour avoir une idée de la complexité et de la charge d'entraînement.

6 Implémentation du Fine-Tuning de l'Audio Spectrogram Transformer (AST)

Ce chapitre détaille l'implémentation du processus de fine-tuning de l'AST sur notre jeu de données cible (par exemple, GTZAN pour la classification de genres musicaux), en utilisant la librairie PyTorch.

6.1 Configuration des Hyperparamètres et Outils

Nous définissons les outils mathématiques essentiels pour l'optimisation et la mesure de performance du modèle.

- **Fonction de Perte (Criterion)** : Nous utilisons la fonction de perte entropie croisée, standard pour les tâches de classification multi-classes.
- **Précision Mixte (AMP)** : L'utilisation de GradScaler est essentielle pour l'entraînement en précision mixte (*Automatic Mixed Precision* ou AMP), qui permet d'accélérer la convergence et de réduire l'utilisation mémoire sur les GPU.

6.1.1 Stratégie du Taux d'Apprentissage

Le planificateur de taux d'apprentissage CosineAnnealingLR est choisi pour sa capacité à améliorer la généralisation en modulant le taux sur l'ensemble des époques.

Outil	Rôle Spécifique
AdamW	Optimiseur avec décroissance de poids corrigée pour les Transformers.
CosineAnnealingLR	Décroissance cosinusoidale du LR sur NUM_EPOCHS pour atteindre un meilleur mini

TABLE 1 – Résumé des outils d’optimisation.

6.2 Fonction d’Évaluation (Validation)

La fonction `evaluate` permet de mesurer l’exactitude (*accuracy*) du modèle sur le jeu de validation sans mise à jour des poids.

```

1  def evaluate(model, loader):
2      model.eval()
3      all_preds, all_labels = [], []
4
5      with torch.no_grad():
6          for X, y in loader:
7              X, y = X.to(DEVICE), y.to(DEVICE)
8              out = model(X)
9              preds = torch.argmax(out, dim=1)
10             all_preds += preds.cpu().tolist()
11             all_labels += y.cpu().tolist()
12
13     acc = accuracy_score(all_labels, all_preds)
14     return acc, np.array(all_labels), np.array(all_preds)
15

```

FIGURE 4 – La fonction `evaluate`.

6.3

7 test Accuracy

```

/tmp/ipython-input-1421155.py:16: UserWarning: PySoundFile failed. Trying audioread instead.
  audio, sr = librosa.load(path, sr=self.sr)
/usr/local/lib/python3.12/dist-packages/librosa/core/audio.py:184: FutureWarning: librosa.core.audio.__audioread_load
  Deprecated as of librosa version 0.10.0.
  It will be removed in librosa version 1.0.
  y, sr_native = __audioread_load(path, offset, duration, dtype)
[CORRUPTED FILE] Skipping /content/drive/MyDrive/genres_original/jazz/jazz.00054.wav:
test Accuracy: 86.67%

```

FIGURE 5 – test Accuracy.

8 Comparaison entre les modèles CNN et AST

Dans cette étude, deux approches de classification musicale ont été évaluées sur le jeu de données *GTZAN* : un réseau de neurones convolutionnel (CNN) classique et le modèle *Audio Spectrogram Transformer* (AST). Les deux modèles utilisent exactement le même prétraitement des données (spectrogrammes log-Mel), la même division des données (train / validation / test) et les mêmes métriques d'évaluation, afin d'assurer une comparaison équitable.

8.1 Principe de fonctionnement des modèles

8.1.1 CNN (Convolutional Neural Network)

Le CNN traite les spectrogrammes comme des images bidimensionnelles. Les couches convolutionnelles permettent de détecter des motifs locaux tels que les harmoniques, les transitoires et les variations de fréquences dans le temps. L'apprentissage est effectué uniquement à partir du jeu de données GTZAN, sans connaissance préalable du domaine audio.

Cependant, les CNN sont principalement adaptés à la capture de dépendances locales et peinent à modéliser des relations globales à long terme, ce qui est crucial dans l'analyse musicale où la structure temporelle joue un rôle important.

8.1.2 AST (Audio Spectrogram Transformer)

Le modèle AST repose sur l'architecture Transformer, initialement conçue pour le traitement du langage naturel. Les spectrogrammes sont découpés en *patches* et traités comme une séquence, permettant au mécanisme d'auto-attention de capturer des dépendances globales sur l'ensemble du signal audio.

De plus, AST bénéficie d'un pré-entraînement sur de larges bases de données audio telles que *AudioSet*, ce qui lui permet de disposer de représentations audio riches et généralisables avant le fine-tuning sur GTZAN.

8.2 Résultats expérimentaux

Les résultats obtenus montrent une différence significative entre les deux approches :

- Le modèle CNN atteint une accuracy d'environ **48–50%** sur l'ensemble de test.
- Le modèle AST obtient une accuracy significativement plus élevée, démontrant une meilleure capacité de généralisation.

Cette amélioration est principalement attribuée à la capacité du Transformer à capturer des relations temporelles longues et à l'utilisation du pré-entraînement AudioSet.

8.3 Analyse comparative

Critère	CNN	AST
Type d'architecture	Convolutionnelle	Transformer (auto-attention)
Capture des dépendances	Locales	Globales (long terme)
Pré-entraînement	Non	Oui (ImageNet + AudioSet)
Performance sur GTZAN	Moyenne ($\approx 50\%$)	Élevée
Besoin en données	Important	Plus robuste aux petits data-sets
Complexité du modèle	Faible à moyenne	Élevée
Temps d'entraînement	Rapide	Plus lent

TABLE 2 – Comparaison entre CNN et AST pour la classification musicale

8.4 Avantages et limitations

8.4.1 Avantages du CNN

- Architecture simple et rapide à entraîner.
- Faible consommation mémoire.
- Bonne interprétabilité des filtres convolutionnels.

8.4.2 Limitations du CNN

- Difficulté à modéliser les dépendances temporelles longues.
- Sensible au sur-apprentissage sur les petits jeux de données.
- Performances limitées sans pré-entraînement.

8.4.3 Avantages du AST

- Capture efficace des relations globales dans le spectrogramme.
- Bénéficie du pré-entraînement sur de larges bases audio.
- Meilleure généralisation sur des jeux de données de taille limitée.

8.4.4 Limitations du AST

- Coût computationnel élevé.
- Besoin de ressources GPU importantes.
- Temps d'entraînement plus long.

8.5 Discussion

Les résultats obtenus montrent clairement que le modèle AST est mieux adapté à la classification de genres musicaux dans le contexte du jeu de données GTZAN. Son mécanisme d'auto-attention lui permet de mieux exploiter la structure temporelle globale des signaux audio, contrairement au CNN qui se limite à des informations locales.

Ainsi, bien que le CNN constitue une solution simple et efficace pour des systèmes embarqués ou des ressources limitées, le modèle AST représente une approche plus performante pour des applications nécessitant une haute précision.